"For no good tree bears bad fruit, nor again does a bad tree bear good fruit, for each tree is known by its own fruit." Luke 6:43-44

Supervised Learning
Classification: DECISION TREE

Course instructor: Prasanna Ranjith Christodoss
Associate Professor of Comp. Sc., Messiah University, PA 17055

Outline

- Decision Tree as classifier
- Induction of DT
- How much information in an attribute
- Coding DT with Python

Decision Tree as a classifier

- Decision trees are a type of supervised learning algorithm used for both classification and regression tasks.
- Decision tree learning is a popular algorithm for classification tasks.
- Decision trees are known for their efficiency in both training and prediction.

The strengths and purposes of decision trees:

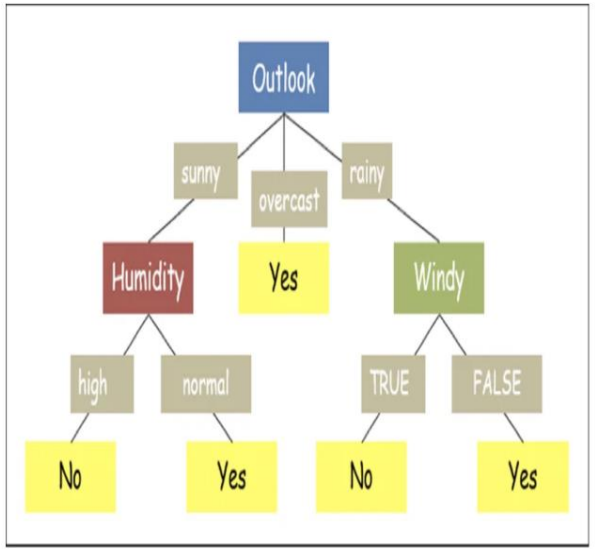
- Multi-dimensional Analysis with Multiple Classes
- Easy Interpretation and Visualization
- Predictive Modeling from Historical Data(past vector)

Decision Tree as a classifier …….

- A decision tree consists of **three types** of nodes:
    - Root Node
    - Branch Node
    - Leaf Node

- Each node in a decision tree corresponds to one feature in the feature vector.
- From every node, there are edges leading to child nodes, with each edge representing a possible value or range of values for the associated feature.

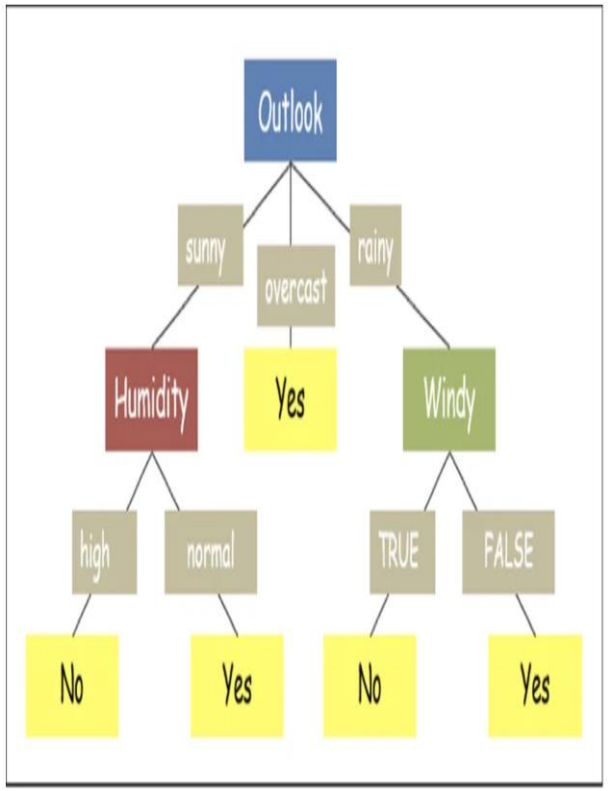
- The tree **terminates at leaf nodes**, also known **as terminal nodes**. Each leaf node represents a possible value for the output variable.



To make a prediction:

- Start at the root node.
- Follow a path guided by the values of the input variables
- Until you reach a leaf node.

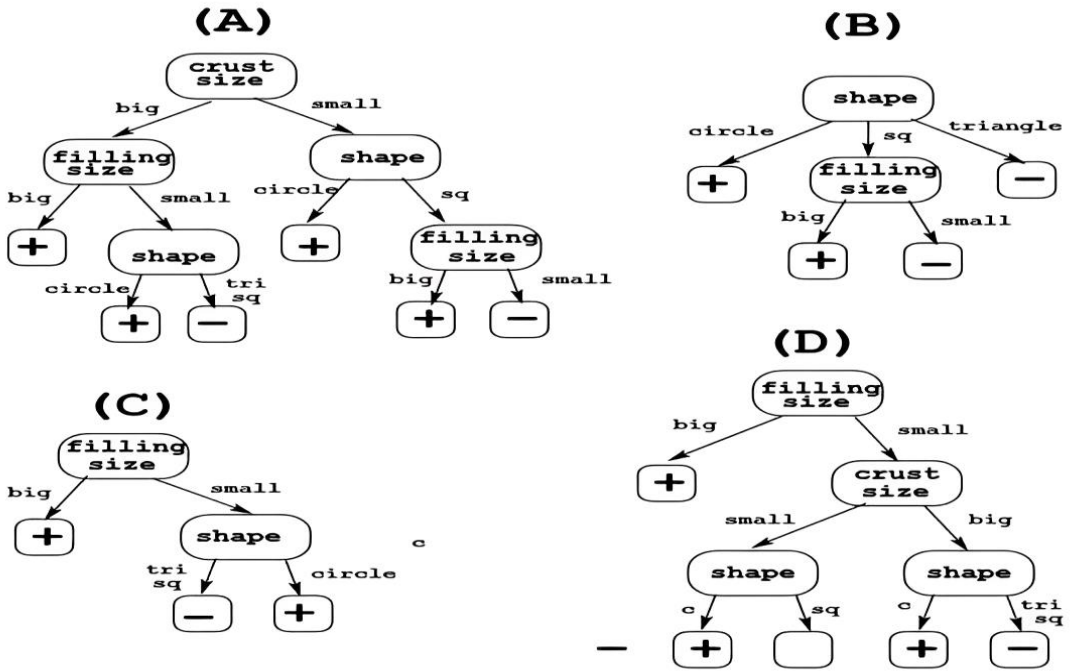
The value associated with that leaf node becomes the prediction for the output variable.



Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

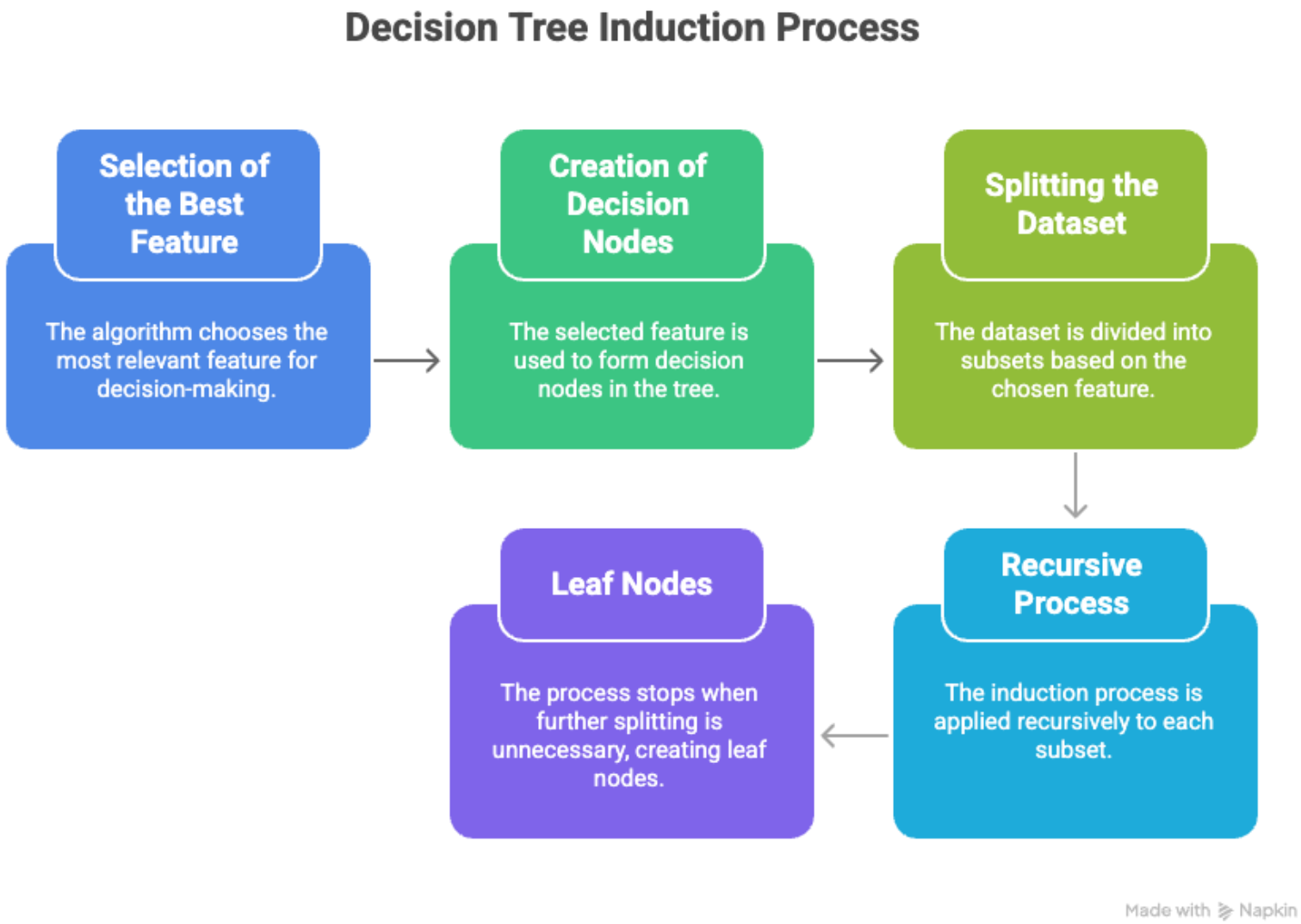
DT Example (Table 1)

Example	crust size	shape	filling size	Class
$e1$	big	circle	small	**pos**
$e2$	small	circle	small	**pos**
$e3$	big	square	small	**neg**
$e4$	big	triangle	small	**neg**
$e5$	big	square	big	**pos**
$e6$	small	square	small	**neg**
$e7$	small	square	big	**pos**
$e8$	big	circle	big	**pos**



- The table above displays various instances classified into positive or negative classes based on their feature values.
- The figure illustrates multiple decision trees constructed from the same dataset, with each decision tree originating from a different feature.

Induction of Decision Tree



Induction of Decision Tree Using Divide and Conquer

The goal of the induction process is to create a decision tree that can make accurate predictions on new, unseen data.

The manual construction of decision trees using the traditional divide-and-conquer method involves making choices at each step to split the dataset based on certain attributes.

Suppose we want to create a decision tree to classify shapes into two categories: "Positive" and "Negative" based on certain attributes.

Example	crust size	shape	filling size	Class
$e1$	big	circle	small	**pos**
$e2$	small	circle	small	**pos**
$e3$	big	square	small	**neg**
$e4$	big	triangle	small	**neg**
$e5$	big	square	big	**pos**
$e6$	small	square	small	**neg**
$e7$	small	square	big	**pos**
$e8$	big	circle	big	**pos**

Example, Eight training examples described by three discrete attributes and classified as positive and negative examples of a given class.

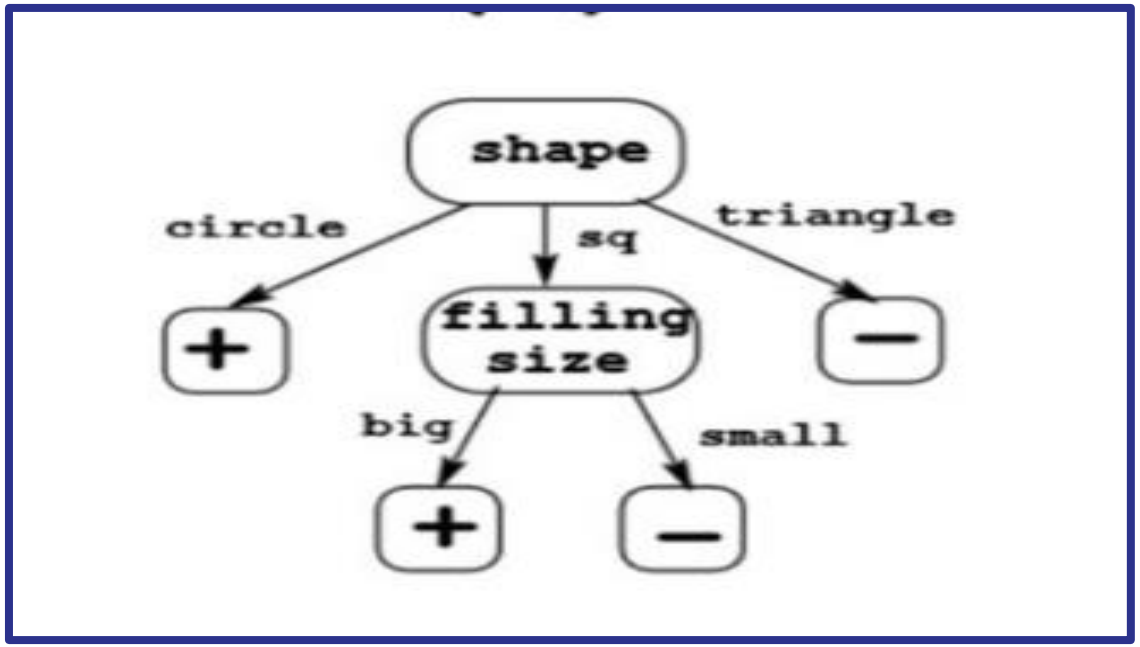
Induction of Decision Tree Using Divide and Conquer ……

Suppose we decide that the root node should test the value of shape

Result Interpretation and Visualization:

Initial Splitting, Attribute Chosen: Shape

- ❑ Subsets Created:
- ❑ Circle Subset (TC): {e1, e2, e8} → Labeled as "Positive"
- ❑ Triangle Subset (TT): {e4} → Labeled as "Negative"
- ❑ Square Subset (TS): {e3, e5, e6, e7}



Example	crust size	shape	filling size	Class
*e*1	big	circle	small	**pos**
*e*2	small	circle	small	**pos**
*e*3	big	square	small	**neg**
*e*4	big	triangle	small	**neg**
*e*5	big	square	big	**pos**
*e*6	small	square	small	**neg**
*e*7	small	square	big	**pos**
*e*8	big	circle	big	**pos**

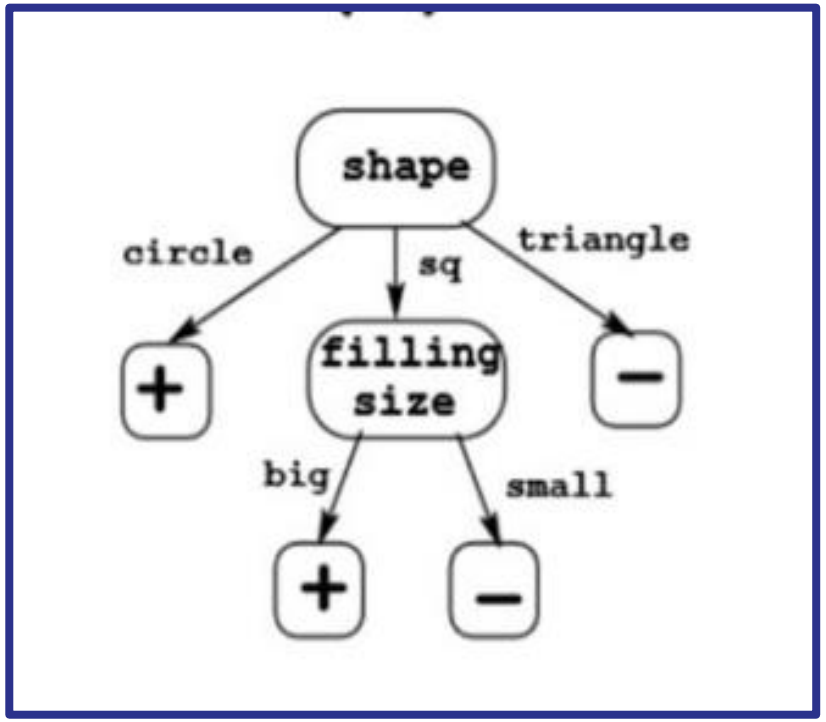
Induction of Decision Tree Using Divide and Conquer ……

Further Splitting (TS Subset)Attribute Chosen: Filling-Size

Subsets Created:

Small Filling-Size (TS−S): {e3, e6} → Labeled as " Negative "

Big Filling-Size (TS−B): {e5, e7} → Labeled as " Positive "



Example	crust size	shape	filling size	Class
e1	big	circle	small	pos
e2	small	circle	small	pos
e3	big	square	small	neg
e4	big	triangle	small	neg
e5	big	square	big	pos
e6	small	square	small	neg
e7	small	square	big	pos
e8	big	circle	big	pos

How to Build a Decision Tree

Recursive Procedure for Decision Tree Induction:

Let T be the training set.

grow(T):

(1) Find the attribute, *at*, that contributes the maximum information about the class labels.
(2) Divide T into subsets, T_i, each characterized by a different value of *at*.
(3) For each T_i:
If all examples in T_i belong to the same class, create a leaf labeled with this class; otherwise, apply the same procedure recursively to each training subset: *grow*(T_i).

- Numerous decision trees can be created from a set of attributes.
- Some trees are more accurate than others.
- Finding the optimal tree mathematically is impossible due to the vast search space.
- Despite the complexity, algorithms exist to generate reasonably accurate decision trees within a reasonable time.
- These algorithms typically use a greedy strategy, making locally optimum decisions about which attribute to use for partitioning the data.

Induction of Decision Tree Using Divide and Conquer

The manual construction process lacks a systematic approach to decide which attributes to choose at each step, leading to different potential trees.

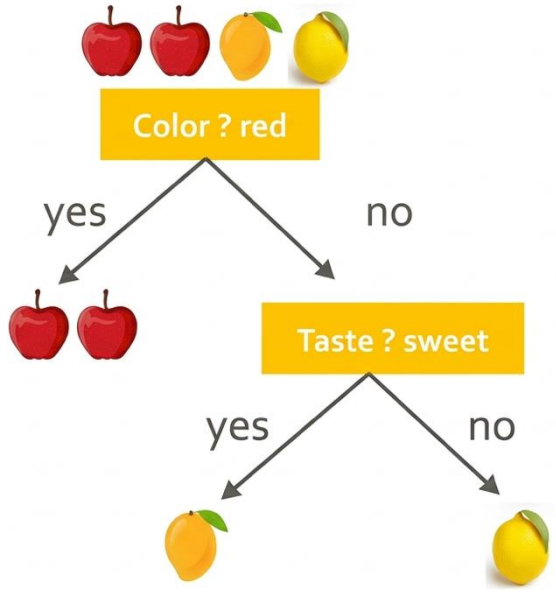
- When manually building decision trees, we noticed that certain attributes provide more information and result in smaller trees.
- For example, "shape" can directly classify examples, while "crust-size" needs assistance from another attribute.
- So, the importance of measuring information from attributes for efficient decision tree induction must be addressed.

Ski-kit learn's 2 Method

- GINI
- ENTROPY

GINI

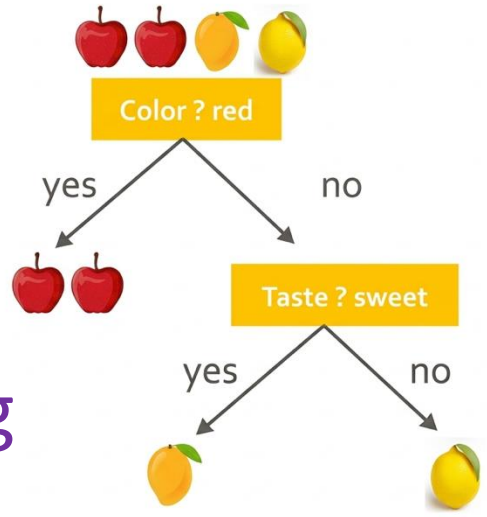
Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon



How to measure the QUALITY of split?

GINI – Shows the probability of misclassifying the dataset

GINI



How to measure the QUALITY of split?

GINI – Shows the probability of misclassifying the dataset

$$\text{GINI Impurity} = \sum_{i-1}^{c} p(i) * (1 - p(i))$$

Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

GINI Gain: How much impurity was reduced before and after split?

$$\text{GINI Gain} = PARENT_{Gini\ Impurity} - (LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$$

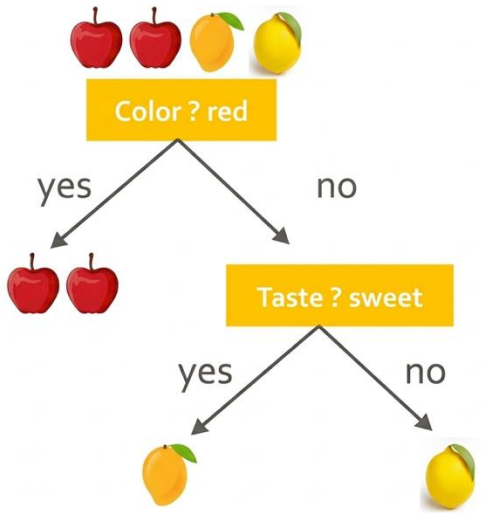
GINI

Step-1: Find Gini Impurity of the whole dataset

$$\text{GINI Impurity} = \sum_{i-1}^{c} p(i) * (1 - p(i))$$

$$= \frac{2}{4}\left[1 - \frac{2}{4}\right] + \frac{1}{4}\left[1 - \frac{1}{4}\right] + \frac{1}{4}\left[1 - \frac{1}{4}\right]$$

$$= 0.625$$



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

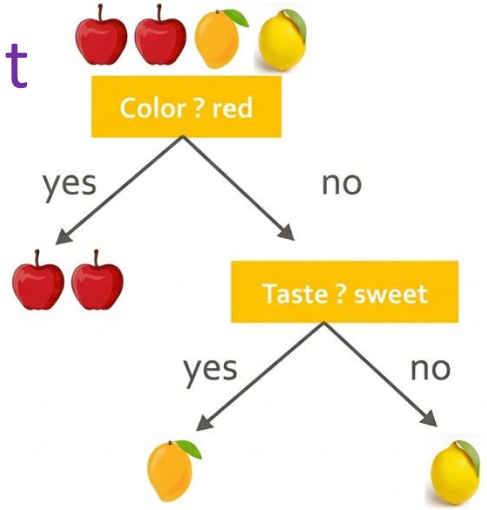
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

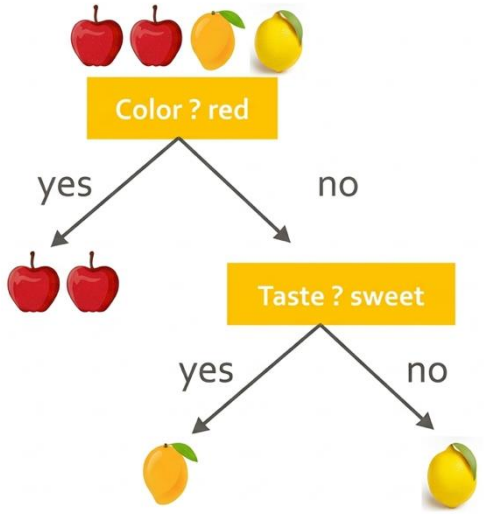
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

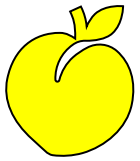
P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

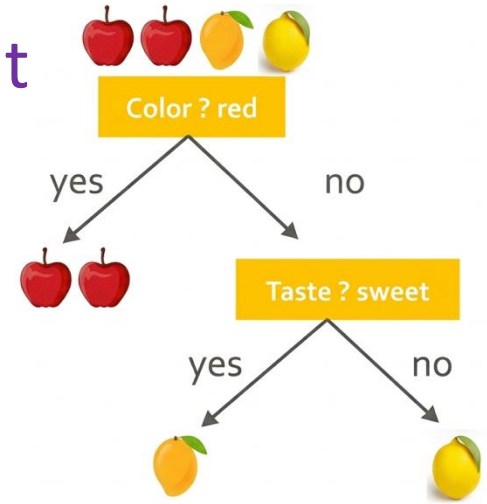
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125



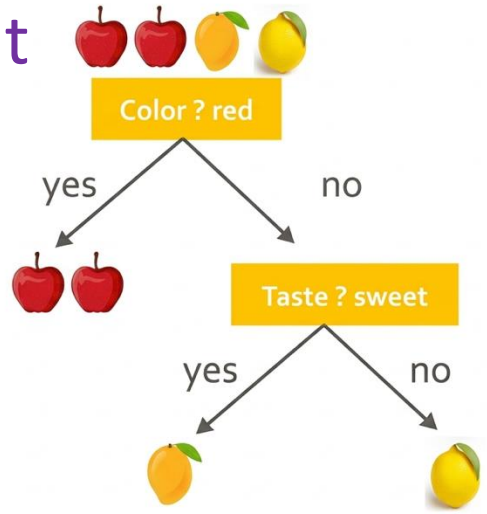
Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

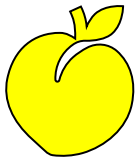
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**



Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Lemon	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625

Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

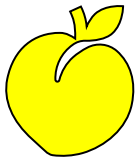
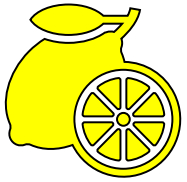
P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

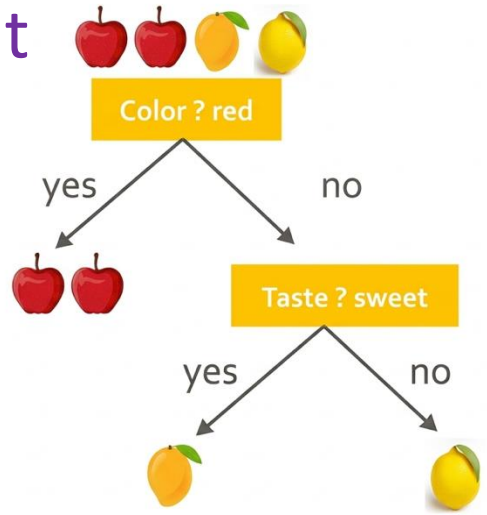
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Lemon	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

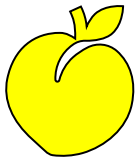
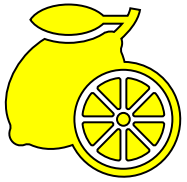
P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

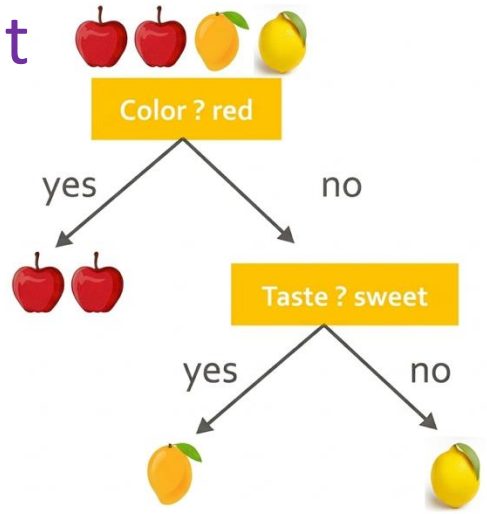
GINI

GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Lemon	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Mango	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

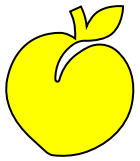
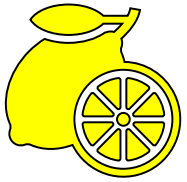
P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

GINI

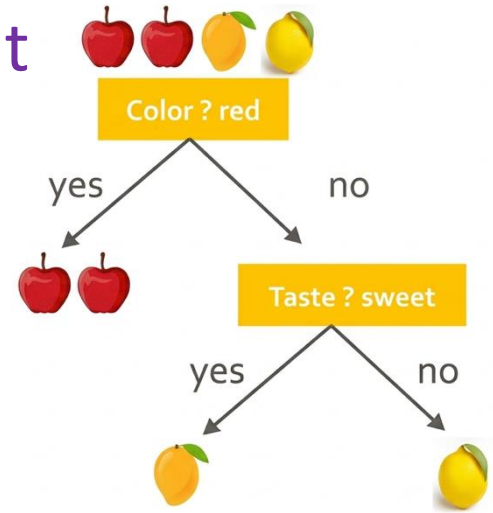
GINI show probability of misclassifying the data point

GINI for whole dataset = **0.625**

Let's try to prove:

Actual Fruit	Wrong label	Probability
	Mango	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Lemon	$\frac{2}{4} \times \frac{1}{4}$ = 0.125
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Lemon	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625
	Apple	$\frac{1}{4} \times \frac{2}{4}$ = 0.125
	Mango	$\frac{1}{4} \times \frac{1}{4}$ = 0.0625

0.625



Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

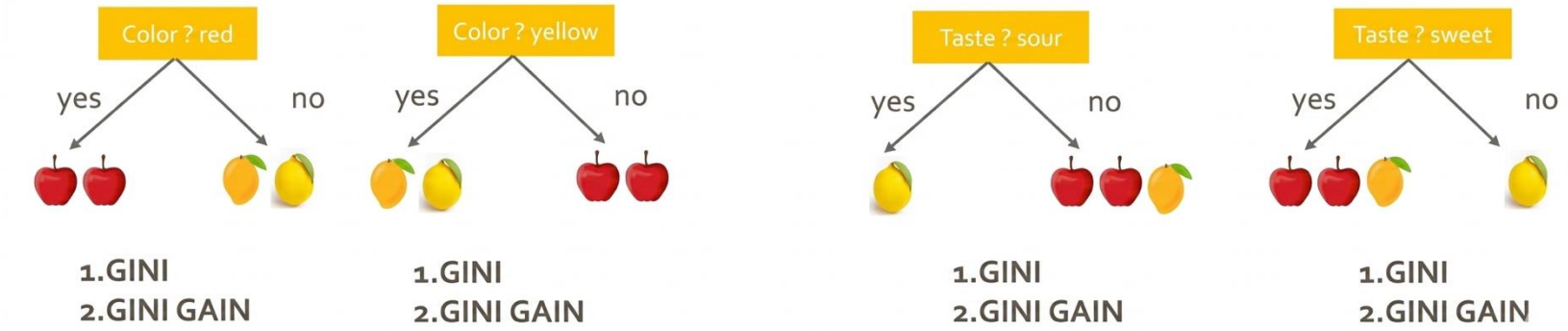
P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

GINI

P(Apple) = 2/4
P(Mango) = 1/4
P(Lemon) = 1/4

Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

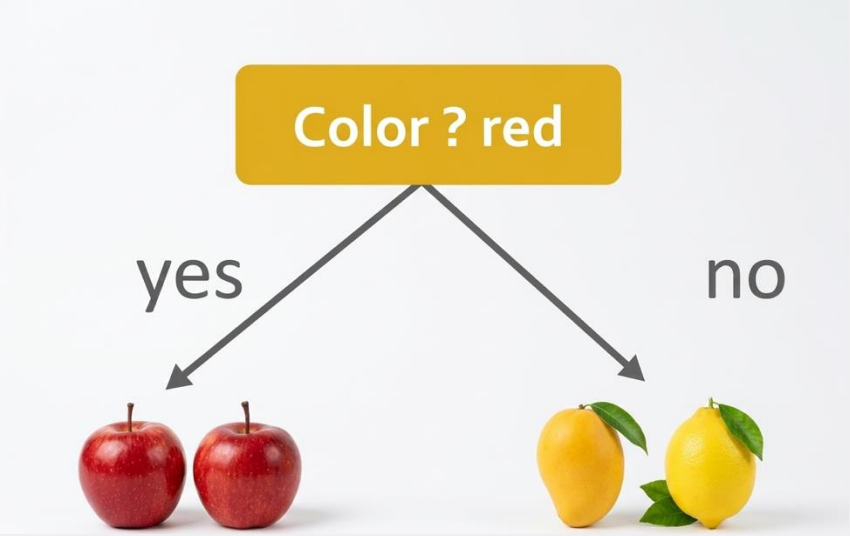
Step-1: Find Gini Impurity of the whole dataset
Step-2: Find the Root Node Candidate



GINI Impurity $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

GINI Gain = $PARENT_{Gini\ Impurity} - (\ LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$

GINI



GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{2}{2}\left[1 - \frac{2}{2}\right] = 0$$

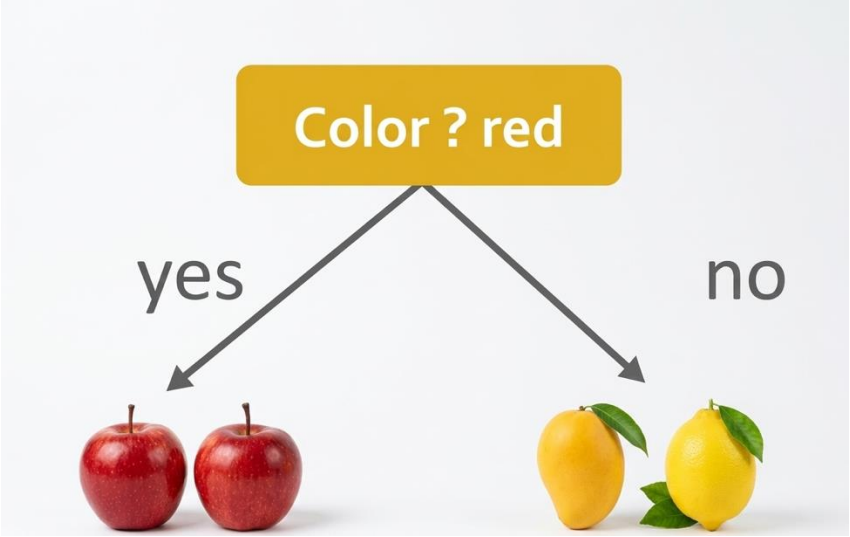
GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{1}{2}\left[1 - \frac{1}{2}\right] + \frac{1}{2}\left[1 - \frac{1}{2}\right] = 0.5$$

GINI

GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{2}{2}\left[1 - \frac{2}{2}\right] = 0$$



GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

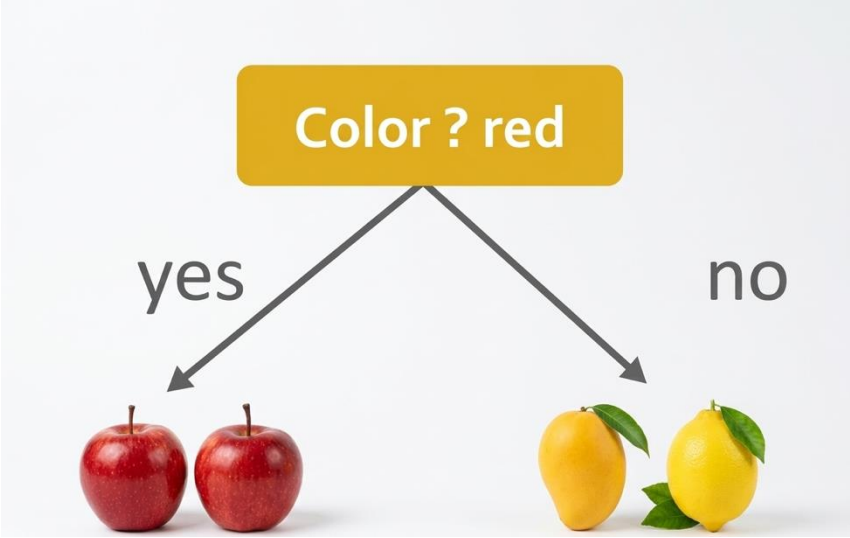
$$\frac{1}{2}\left[1 - \frac{1}{2}\right] + \frac{1}{2}\left[1 - \frac{1}{2}\right] = 0.5$$

WEIGHTED GINI(Left) $= \left(\frac{2}{4}\right)$ X 0 = 0

WEIGHTED GINI(Right) $= \left(\frac{2}{4}\right)$ X 0.5 = 0.25

$$\text{WEIGHTED GINI} = \left(\frac{\text{NO. of sample in current branch}}{Total\ No.of\ Sample\ in\ both\ branch}\right) \text{X Branch GINI Value}$$

GINI



GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{2}{2}\left[1 - \frac{2}{2}\right] = 0$$

GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{1}{2}\left[1 - \frac{1}{2}\right] + \frac{1}{2}\left[1 - \frac{1}{2}\right] = 0.5$$

WEIGHTED GINI(Left) $= \left(\frac{1}{4}\right)$ X 0 = 0

WEIGHTED GINI(Right) $= \left(\frac{2}{4}\right)$ X 0.5 = 0.25

GINI Gain = $0.625 - (0 + 0.25)$
= 0.372

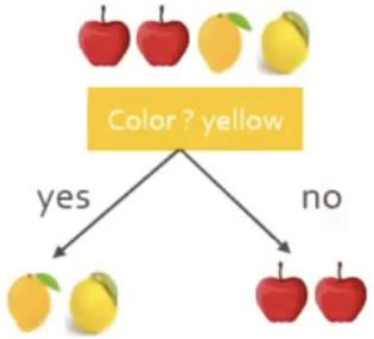
GINI Gain =

$$PARENT_{Gini\ Impurity} - (LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$$

Repeat the same steps to find GINI GAIN of other candidates of root:

a. GINI of Left and Right branch
b. Weighted GINI for Left & Right branch
c. GINI GAIN of entire branch

GINI



$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

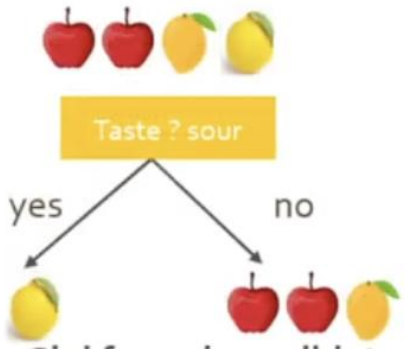
$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{1}{2} * \left(1 - \frac{1}{2}\right) + \frac{1}{2} * \left(1 - \frac{1}{2}\right) = 0.5$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{2}{2} * \left(1 - \frac{2}{2}\right) = 0$$



Gini for 3rd candidate

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

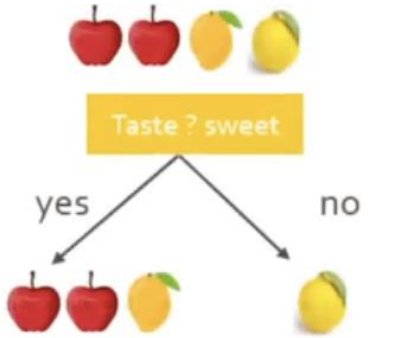
$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{1}{1} * \left(1 - \frac{1}{1}\right) = 0$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{2}{3} * \left(1 - \frac{2}{3}\right) + \frac{1}{3} * \left(1 - \frac{1}{3}\right) = 0.44$$



Gini for 4th candidate

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

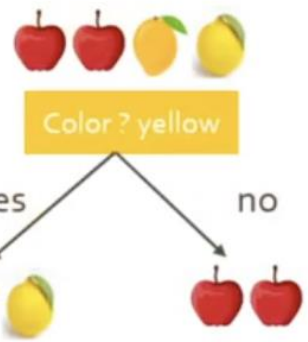
$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{2}{3} * \left(1 - \frac{2}{3}\right) + \frac{1}{3} * \left(1 - \frac{1}{3}\right) = 0.44$$

$$GINI = \sum_{i=1}^{C} p(i) \cdot (1 - p(i))$$

$$\frac{1}{1} * \left(1 - \frac{1}{1}\right) = 0$$

$$\text{WEIGHTED GINI} = \left(\frac{\text{NO. of sample in current branch}}{Total\ No.of\ Sample\ in\ both\ branch}\right) \text{X Branch GINI Value}$$



Weighted Gini for 2nd candidate

GINI value =0.5

2 sample in left

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

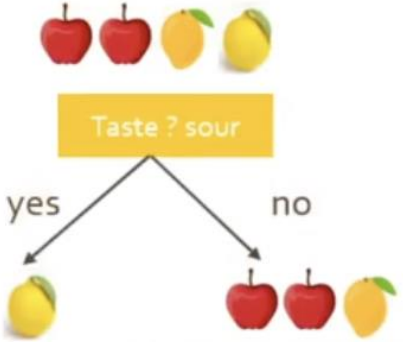
$$\frac{2}{4}*(0.5) \quad = 0.25$$

GINI value =0

2 samples in right

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

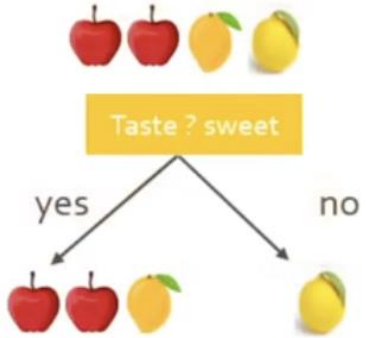
$$\frac{2}{4}*(0) \quad = 0$$



Weighted Gini for 3rd candidate

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

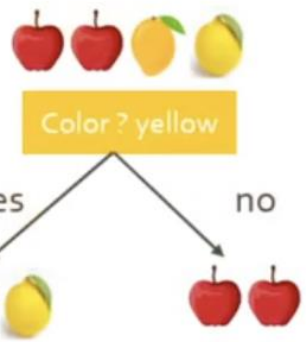


Weighted Gini for 4th candidate

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

$$\text{WEIGHTED GINI} = \left(\frac{\text{NO. of sample in current branch}}{Total\ No.of\ Sample\ in\ both\ branch}\right) \text{X Branch GINI Value}$$



Weighted Gini for 2nd candidate

GINI value =0.5

2 sample in left

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

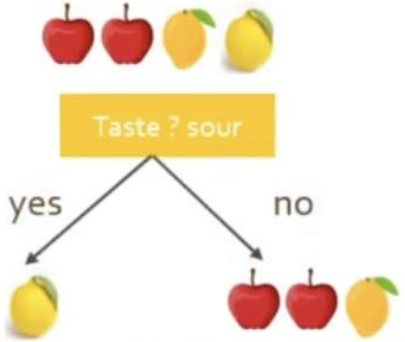
$$\frac{2}{4}*(0.5) = 0.25$$

GINI value =0

2 samples in right

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

$$\frac{2}{4}*(0) = 0$$



Weighted Gini for 3rd candidate

GINI value =0

1samples in left

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

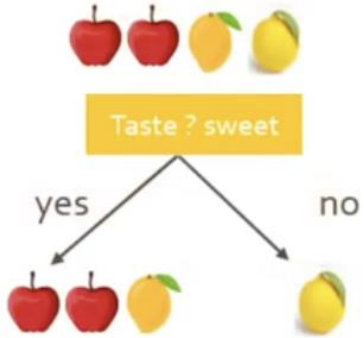
$$\frac{1}{4}*(0) = 0$$

GINI value =0.44

3samples in right

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

$$\frac{3}{4}*(0.44) = 0.33$$



Weighted Gini for 4th candidate

GINI value =0.44

3samples in left

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

$$\frac{3}{4}*(0.44) = 0.33$$

GINI value =0

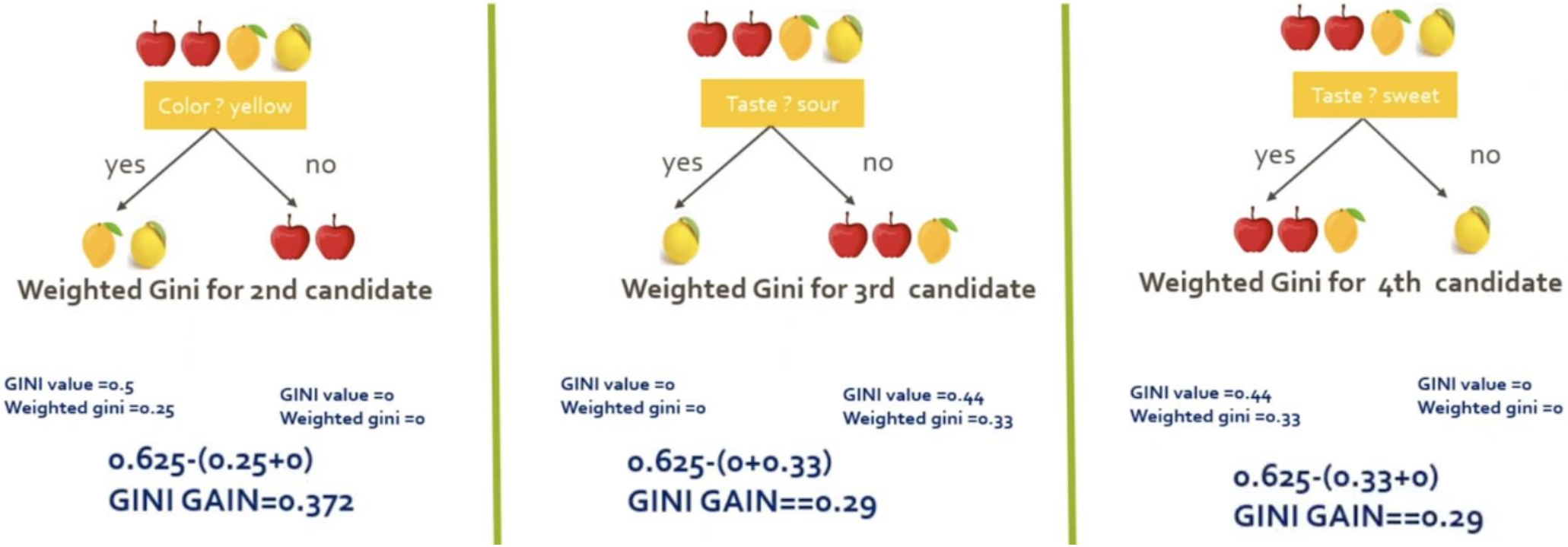
1samples in right

Weighted GINI =(No of sample in current branch/ Total No of sample in both branch)*GINI value

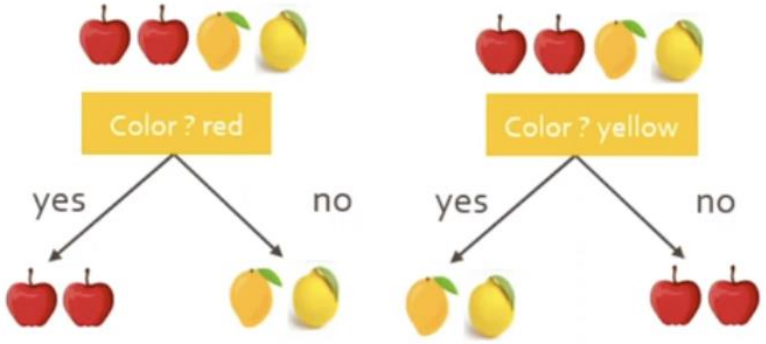
$$\frac{1}{4}*(0) = 0$$

GINI Gain =

$$PARENT_{Gini\ Impurity} - (\ LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$$

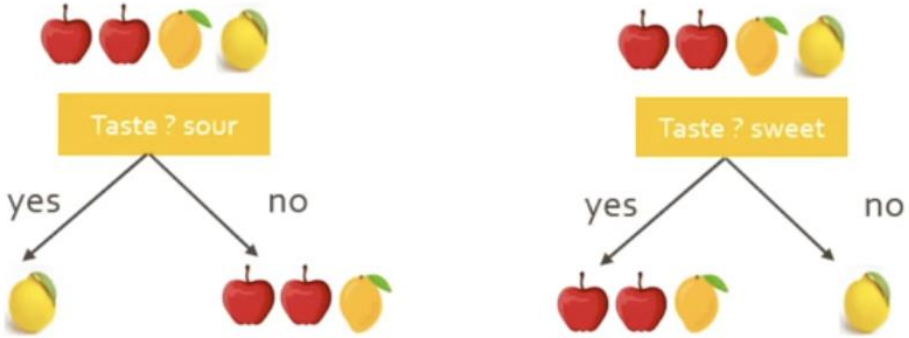


Color ? Red & color ? yellow has same gini gain
which means they did same split
so Pick randomly one candite since they both represent color feature

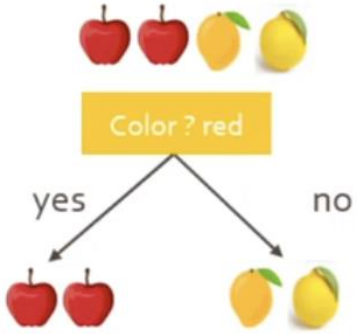


0.372 0.372

Color ? Red & color ? yellow has same gini gain
which means they did same split
so Pick randomly one candite since they both represent color feature

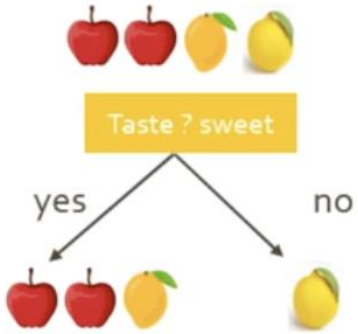


0.29 0.29



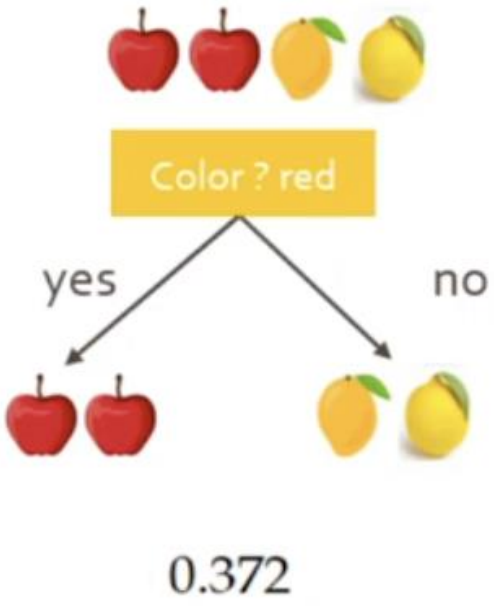
0.372

Chose the split with HIGH GINI GAIN



0.29

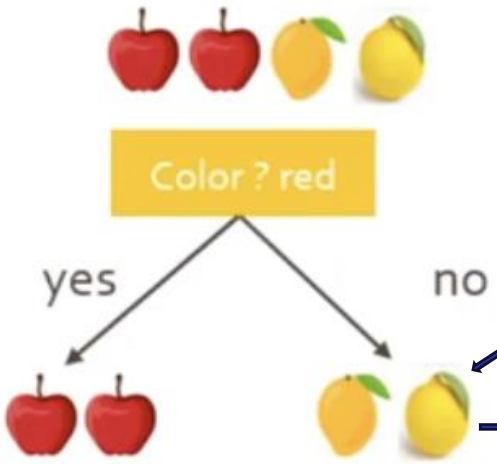
Reduction in impurity before and after the split



Initial GINI GAIN = 0.625

0.372

We have identified the ROOT.



Let's repeat the same on the subset data on the right node!

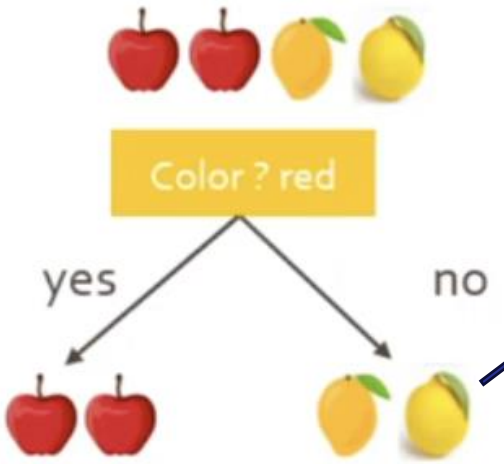
Subset Dataset

Color	Taste	Class
Yellow	Sweet	Mango
Yellow	Sour	Lemon

Original Dataset

Color	Taste	Class
Red	Sweet	Apple
Red	Sweet	Apple
Yellow	Sweet	Mango
Yellow	Sour	Lemon

Splitting the Subset



Subset Dataset

Color	Taste	Class
Yellow	Sweet	Mango
Yellow	Sour	Lemon

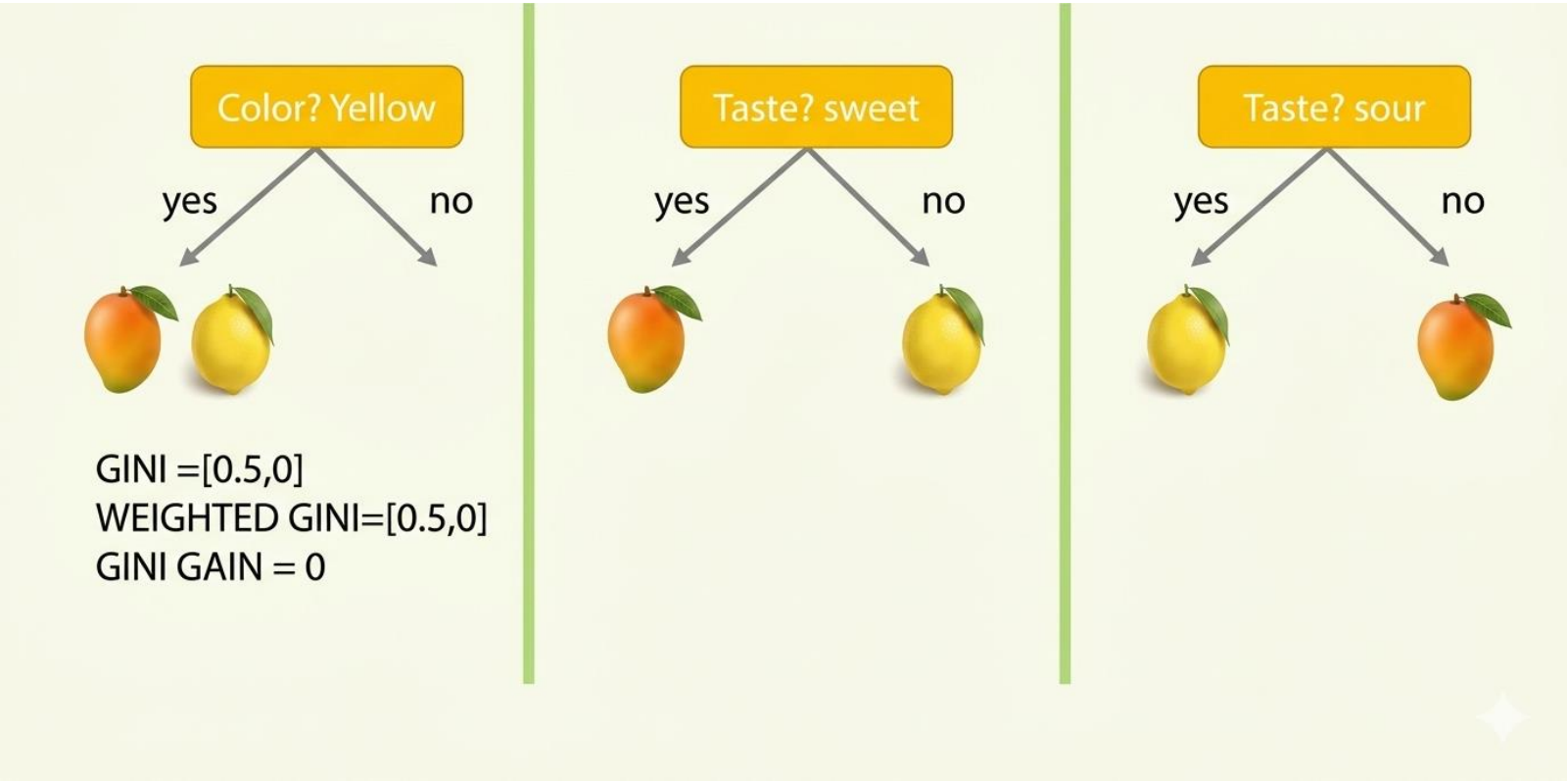
Find the Parent GINI

GINI $= \sum_{i-1}^{c} p(i) * (1 - p(i))$

$$\frac{1}{2}\left[1 - \frac{1}{2}\right] + \frac{1}{2}\left[1 - \frac{1}{2}\right] = 0.5$$

Original Dataset

Splitting the Subset



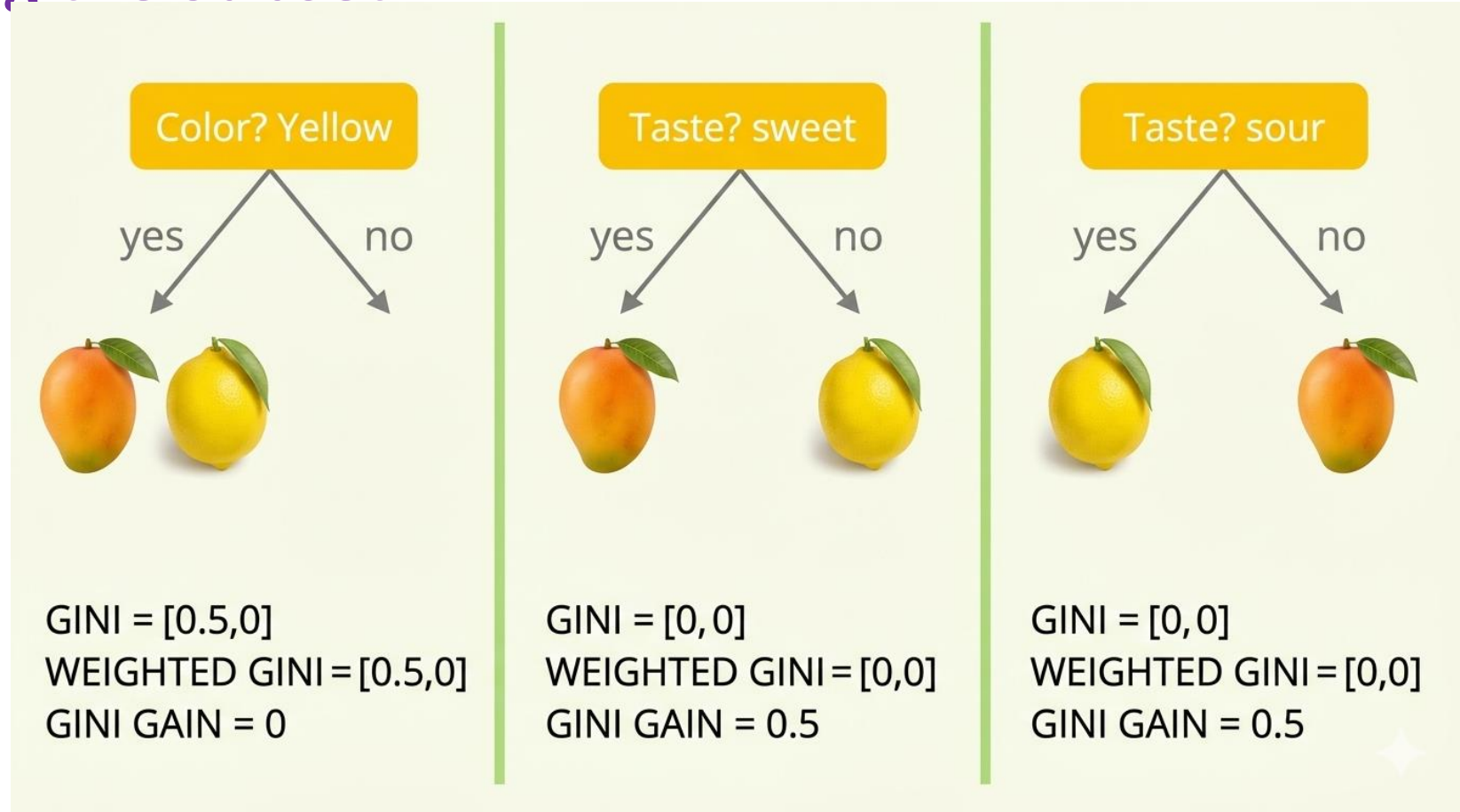
GINI $=\sum_{i-1}^{c} p(i) * (1 - p(i))$

WEIGHTED GINI = $\left(\frac{\text{NO. of sample in current branch}}{Total\ No.of\ Sample\ in\ both\ branch}\right)$ X Branch GINI Value

GINI Gain =

$PARENT_{Gini\ Impurity} - (\ LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$

Splitting the Subset



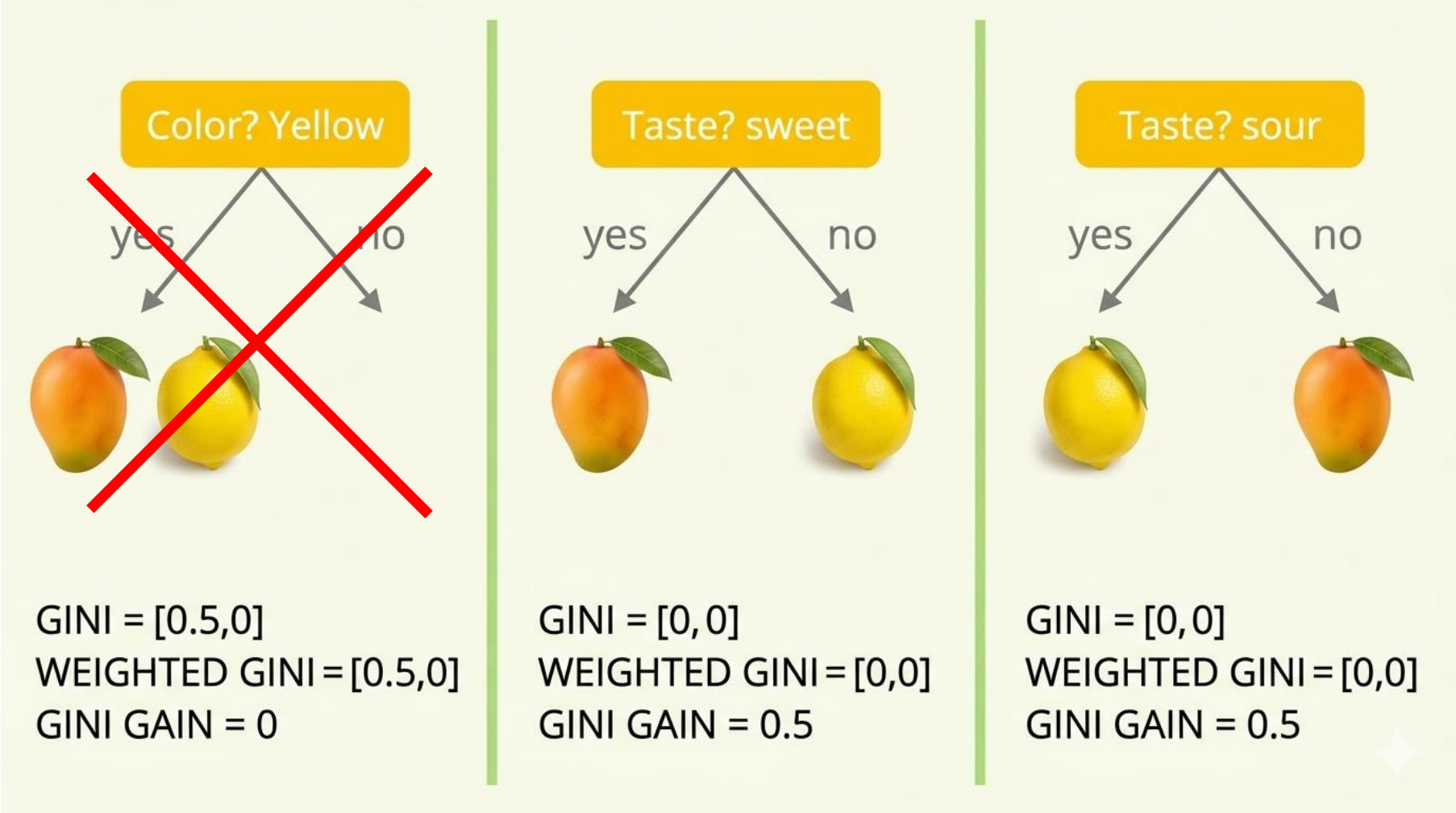
GINI $= \sum_{i-1}^{c} p(i) * (1 - p(i))$

WEIGHTED GINI = $\left(\frac{\text{NO. of sample in current branch}}{Total\ No.of\ Sample\ in\ both\ branch}\right)$ X Branch GINI Value

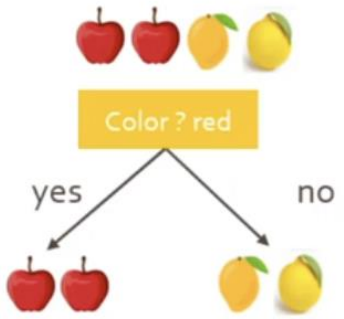
GINI Gain =

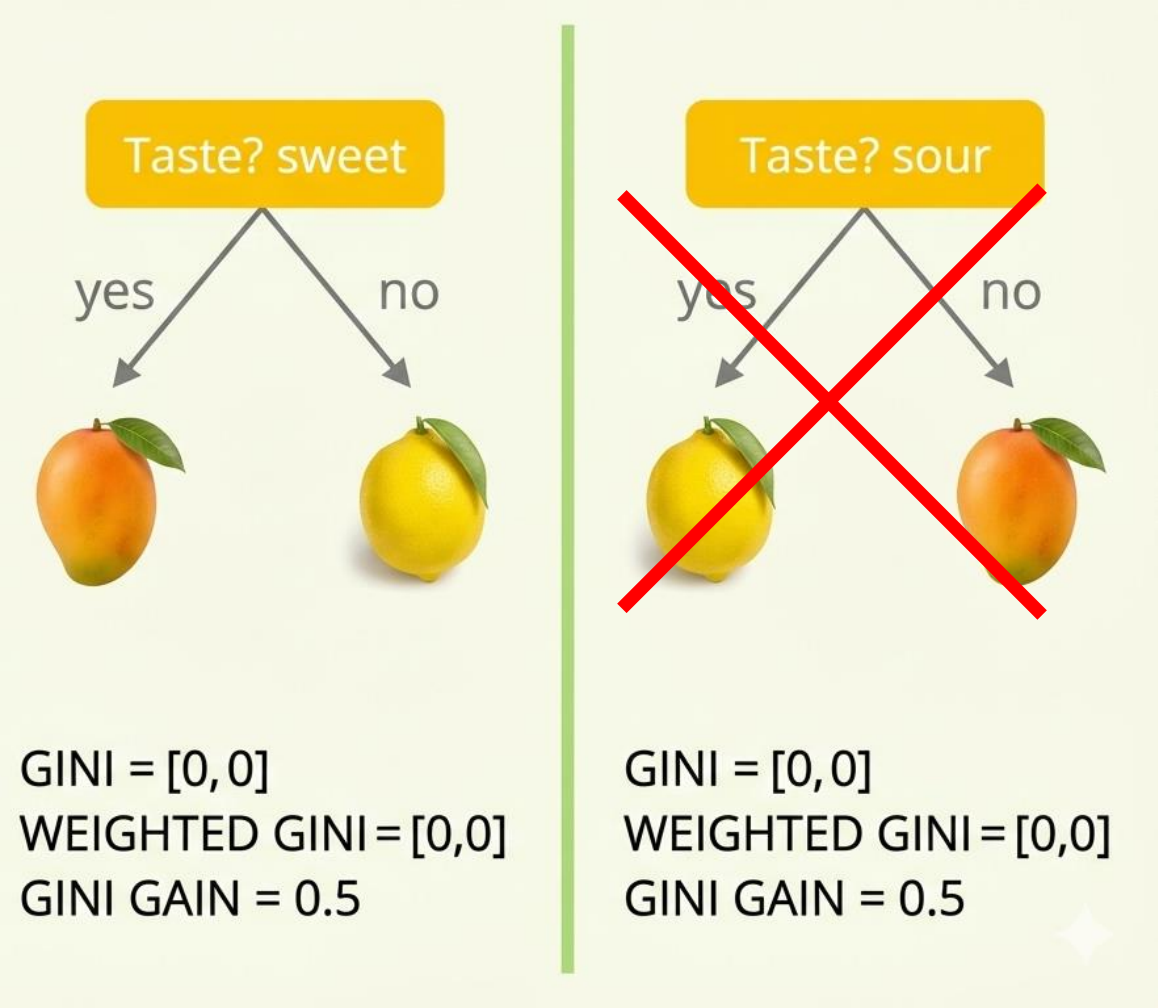
$$PARENT_{Gini\ Impurity} - (LEFT\ NODE_{Weighted\ gini\ Impurity} + RIGHT\ NODE_{Weighted\ gini\ Impurity})$$

Splitting the Subset

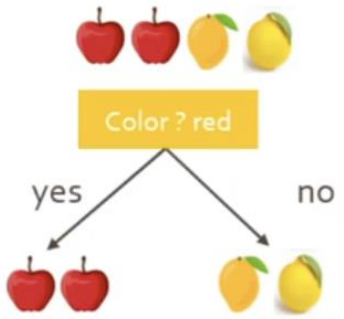


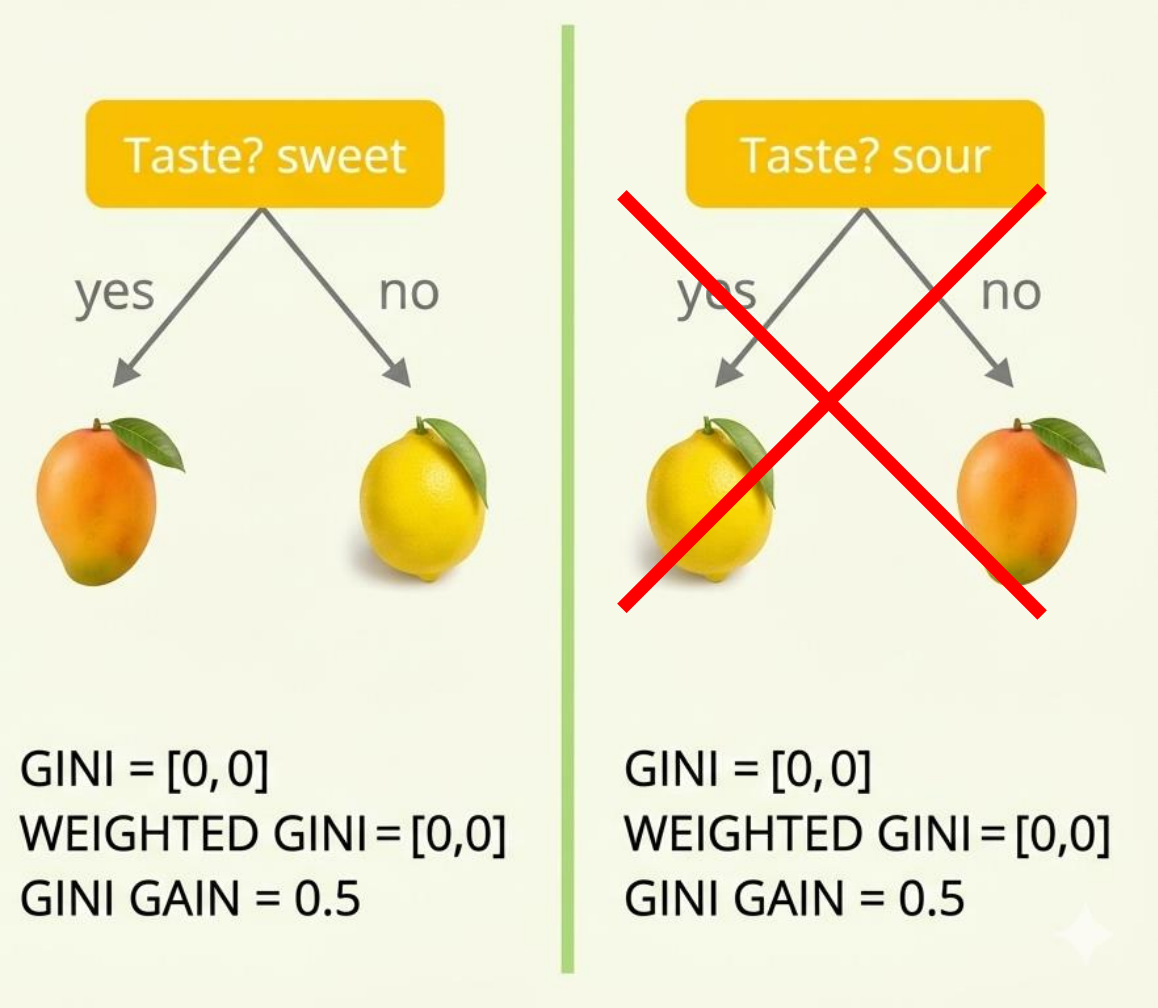
Splitting the Subset



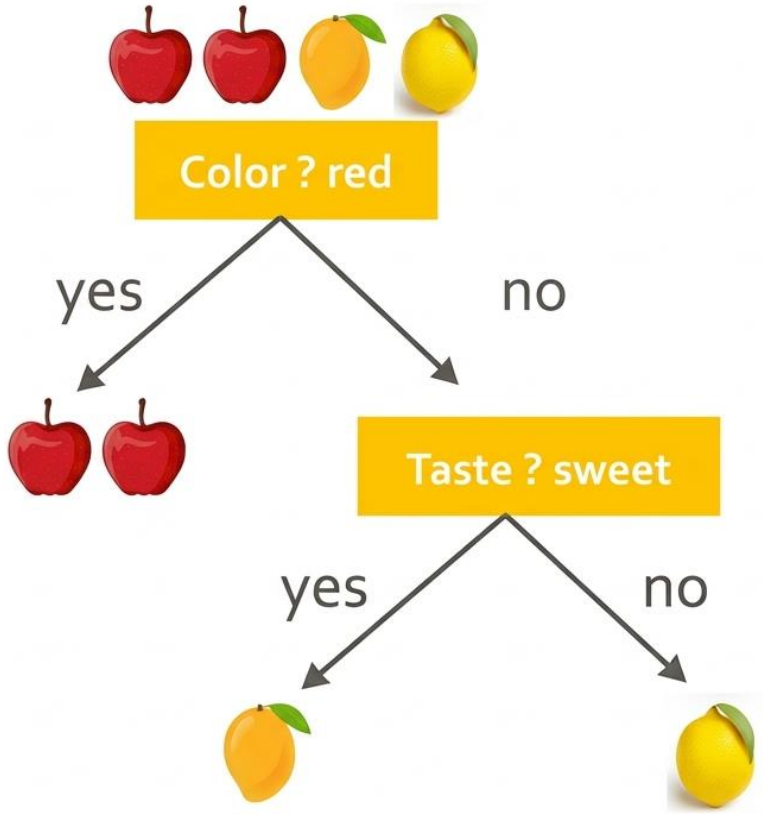


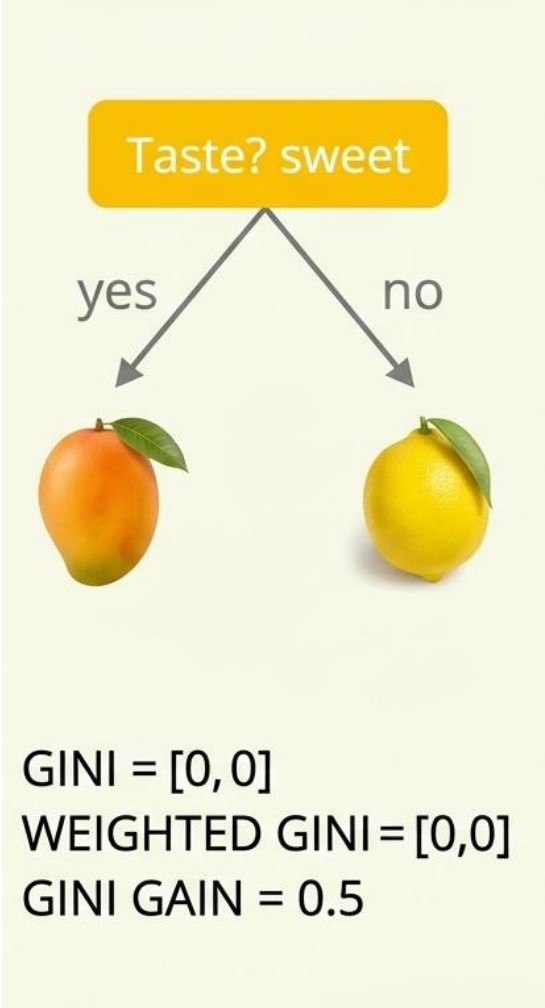
Splitting the Subset





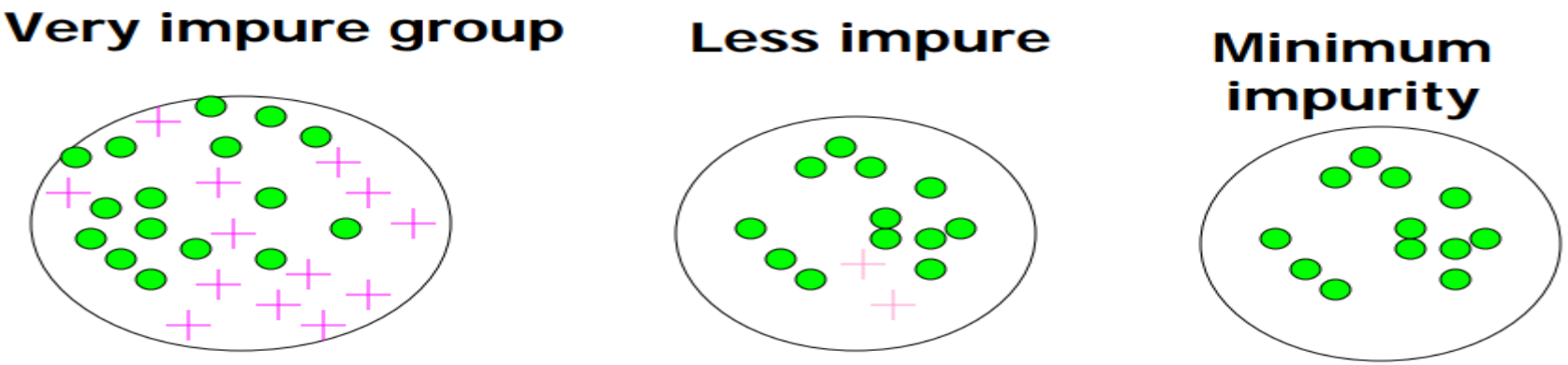
Splitting the Subset





What is Entropy

- Impurity/Entropy (informal) – Measures the level of impurity in a group of examples(instance).
- Entropy is a common way to measure impurity.
- Entropy comes from information theory

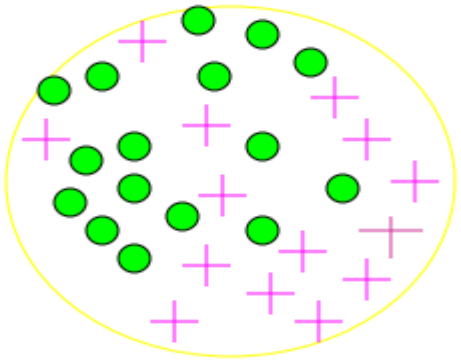


Entropy …….

- Entropy at a given node t:

$$Entropy(t) = -\sum_j p(j \mid t) \log p(j \mid t)$$

(NOTE: $p(j \mid t)$ is the relative frequency of class j at node t).



The probability of class green circles is log2(16/30) =-0.9
While the probability of class pink crosses is log2(14/30) = -1.1;
Entropy = -(16/30)(-.9) –(14/30)(-1.1) = .99

Examples for computing Entropy

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 log 0 – 1 log 1 = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (1/6) = 0.65

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Information Gain

- We want to determine which attribute in a given set of training feature vectors is most useful for discriminating between the classes to be learned.

- Information gain tells us how important a given attribute of the feature vectors is.

- We will use it to decide the ordering of attributes in the nodes of a decision tree.

Information Gain (INFO)...

- Information Gain:

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i) \right)$$

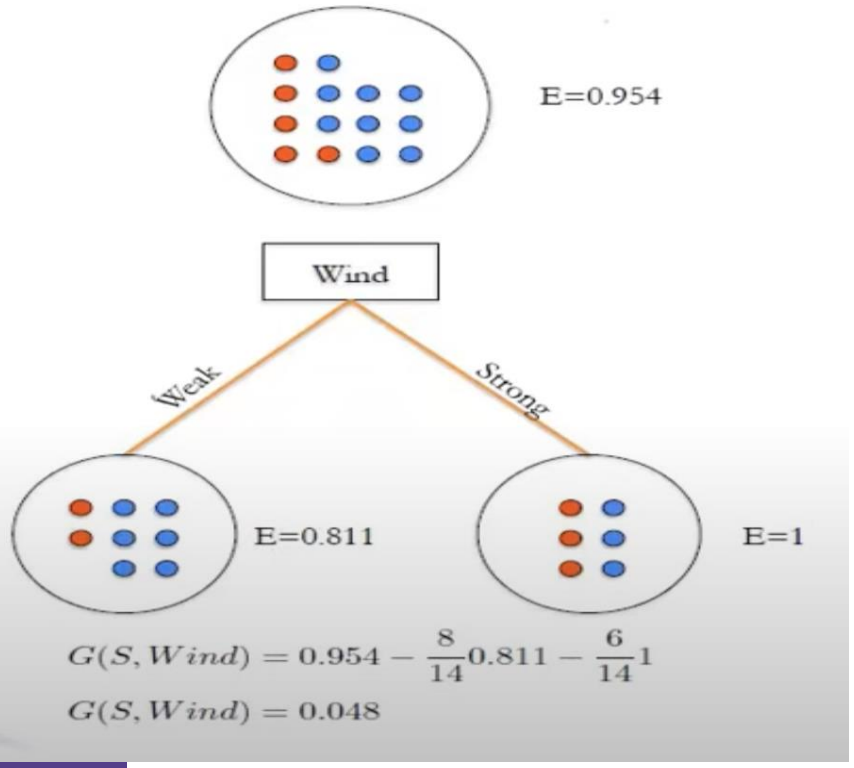
Parent Node, p is split into k partitions;

n_i is number of records in partition i

- **Measures Improvement**
- **Guides Decision Making**
- **Enhances Predictive Accuracy**
- **Efficient Feature Selection**
- **Commonly Used in Decision Trees**

wisdom expertise

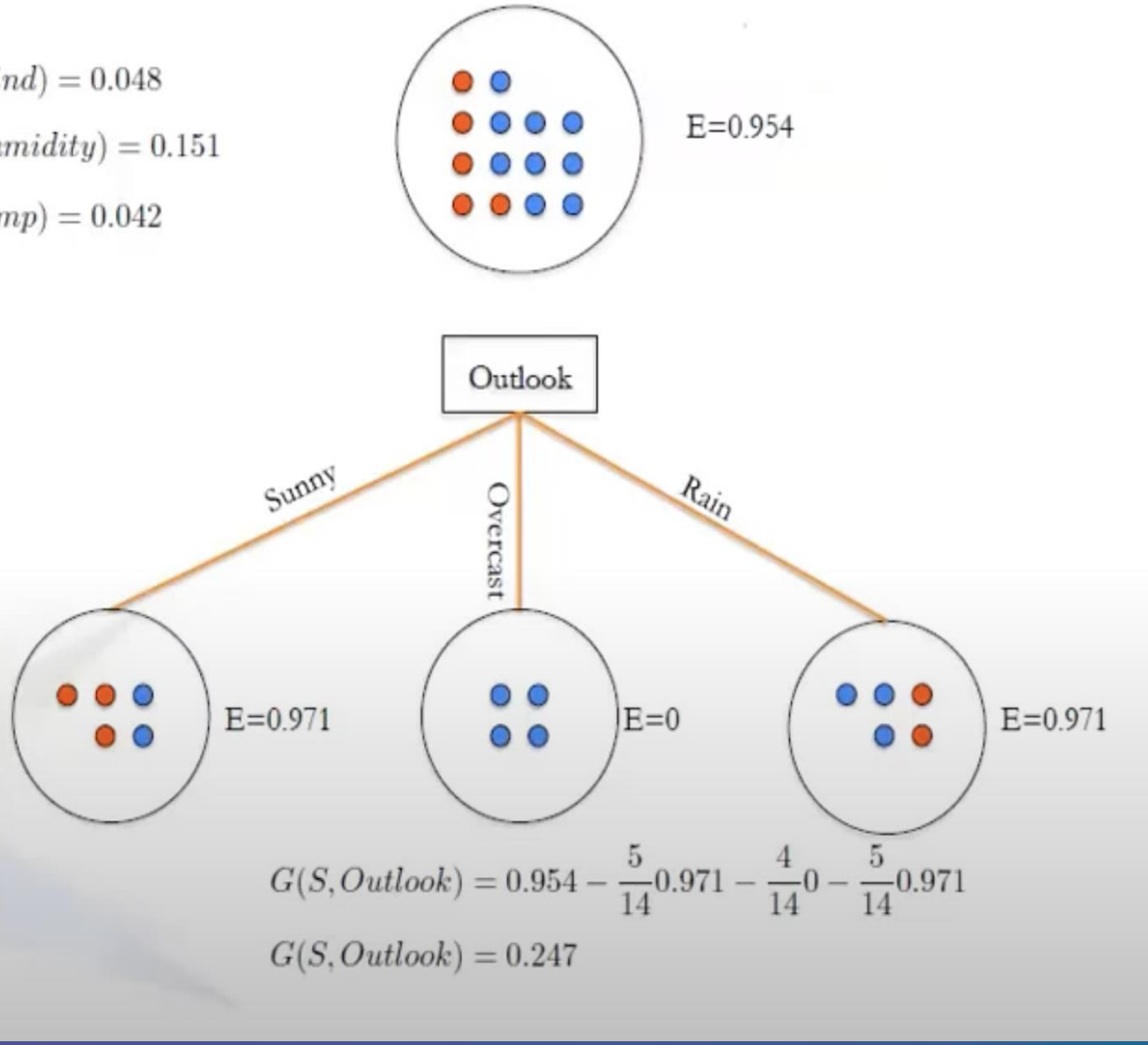
Information Gain (INFO)...



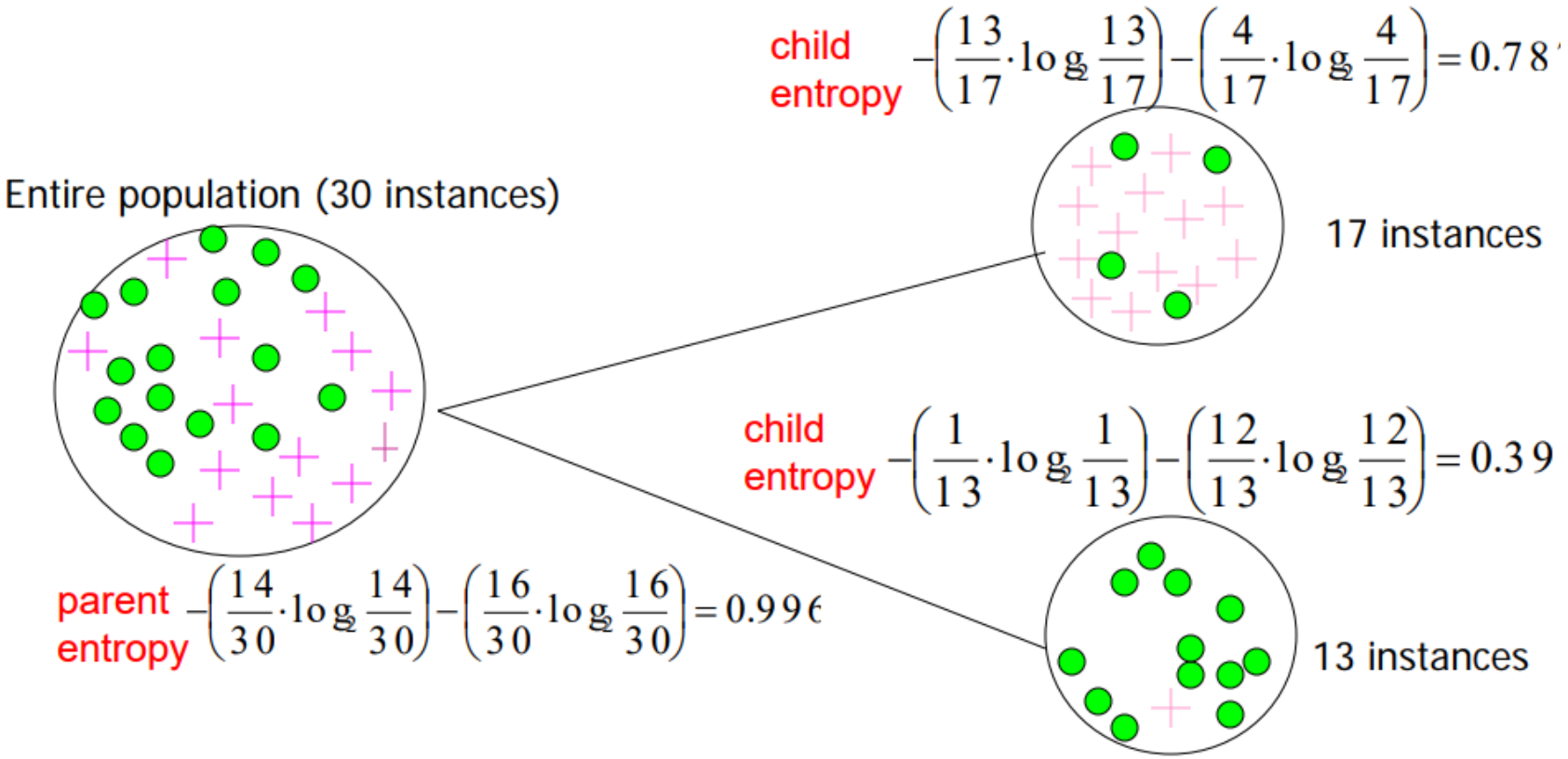
$G(S, Wind) = 0.048$

$G(S, Humidity) = 0.151$

$G(S, Temp) = 0.042$



Example: Calculating Information Gain



- (Weighted) Average Entropy of Children = $\left(\frac{17}{30}\cdot 0.787\right)+\left(\frac{13}{30}\cdot 0.391\right)=0.615$
- Information Gain= 0.996 - 0.615 = 0.38 for this split

DT with python

You are a data scientist working with a botanical research team. The team has collected measurements of iris flowers, and they want to use machine learning to classify iris species based on these measurements. You decide to use a Decision Tree Classifier for this task.

Import Libraries:

import matplotlib.pyplot as plt: Imports the matplotlib library for plotting.

from sklearn import tree: Imports the tree module from the sklearn library for Decision Tree classification.

from sklearn.datasets import load_iris: Imports the Iris dataset from sklearn.

from sklearn.model_selection import cross_val_score: Imports cross-validation functionality from sklearn.

Load the Iris Dataset

```
iris = load_iris()
```

Create Decision Tree Classifier: **Creates an instance of the Decision Tree Classifier called dtc.**

```
dtc = tree.DecisionTreeClassifier()
```

Train the Decision Tree: **Trains the Decision Tree using the entire Iris dataset.**

```
dtc.fit(iris.data, iris.target)
```

Cross-validate the Model:

```
scores = cross_val_score(dtc, iris.data, iris.target, cv=3, scoring='accuracy')
```

Performs cross-validation on the Decision Tree model using the Iris dataset.
cv=3 specifies 3-fold cross-validation.
scoring='accuracy' evaluates the accuracy of the model.

Display Accuracy Scores:

```
print("Cross-validated accuracy scores:", scores)
print("Mean accuracy:", scores.mean())
```

Prints the accuracy scores for each fold and the mean accuracy across all folds.

wisdom expertise